Hive and HBase

Lecture 4

Big Data Management

Francesco Fabbrocino

UCLA Extension

Lecture Roadmap

1. Introduction to Big Data
2. Hadoop and HDFS
3. Hadoop and MapReduce
4. <u>Hive and HBase</u>
5. Introduction to Spark
6. Spark SQL, MLlib, Streaming and GraphX
7. Data Ingestion and Big Data Architecture
8. Big Data in the Cloud
9. The Future of Big Data

Lecture 4 Agenda

- Roll Call
- Lecture 2 Test Results
- Lecture 3 Review and Test
- Lecture 3 Homework Review
- Lecture 4: Hadoop and Hive and HBase
- Lecture 4 Homework

Roll Call

- Per UCLA Extension policy, 10% of your grade is based on in-person attendance.
- *If you are late to class and miss Roll Call, please see me after class ends to update your attendance record.*

Lecture 2 Test Results Review

- Questions?

Lecture 3 Review

- Questions?

Lecture 3 Test

- Closed Book, Notes, Computer and Phone
- Please Remember to Select the best Answer to Each Question
- 12 Multiple Choice Questions
- 15 Minutes (extra)

Lecture 3 Homework Review

- Questions?

Lecture 4 Agenda

- Apache Hive
- Apache HBase

What is Apache Hive?

- Hive is a Data Warehousing Framework that Runs on Hadoop
- Originally Created by Facebook, now an Official Apache Project
- *Hive brought Interactive Queries to Hadoop*

What is a Data Warehouse?

- Data Warehouses are a Special Type of Database Designed for Reporting and Data Analysis of Large Amounts of Data
- Data Warehouses are usually implemented using a Relational Database
- *Data Warehouses provide so-called “Online Analytical Processing” (OLAP) whereas conventional databases that provide so-called “Online Transactional Processing” (OLTP)*

Databases vs Data Warehouses

	Database	Data Warehouse
Technology	Relational + SQL	Relational + SQL
Schema	Normalized	Denormalized
Type	Transactional (OLTP)	Analytical (OLAP)
Data	Original	Copy
Timeframe	Current	Historical
Application	Few	Multiple
Size	Smaller	Larger
Operations	Create, Read, Update, Delete (CRUD)	Read

What Did Hive do for Hadoop?

- Hive provides a SQL-like Interface to Data in Hadoop
- Hive SQL is a Subset of ANSII SQL Called “HiveQL” or “HQL”
- HQL Can be Developed and Executed via a Command Line Interface or “CLI”
- Rather than Develop MapReduce Jobs, Data in Hadoop can be Interactively Analyzed using HQL
- *Hive has limited support for constraints and other features when compared to Relational Databases*

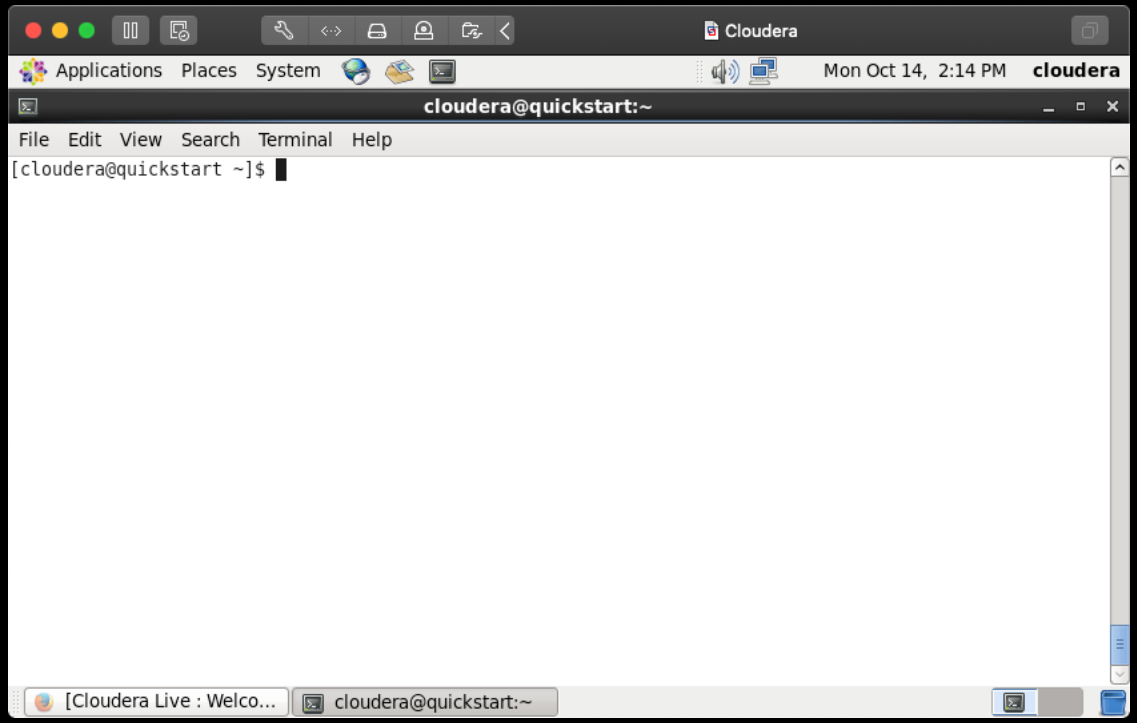
How Does Hive Work?

1. Data is Imported into Schemas in Hive that are Similar to a Traditional Database
2. Data is Stored in HDFS with Metadata in Hive
3. Hive Users Write and Execute Code written in HQL
4. Hive Executes the HQL Code by Converting it into MapReduce Jobs Automatically

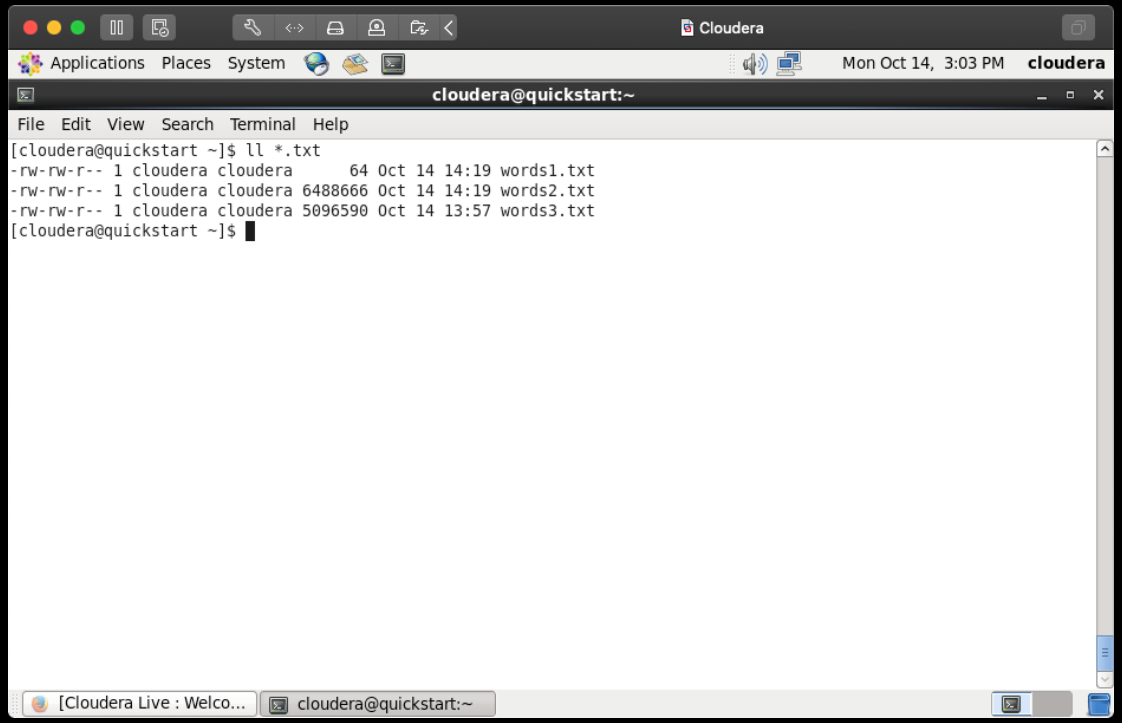
“Schema on Read” versus “Schema on Write”

- Conventional Databases require that a Schema be defined before data can be ingested into it; This is known as, “Schema on Write”
- “Schema on Read” is where the schema is defined automatically based on the data ingested, so you do not need to define the schema first
- Hive supports both “Schema on Write” and “Schema on Read”
- Schema on Write is predominantly used in OLTP; Schema on Read is typically used in OLAP

Hive Demo: Open Terminal



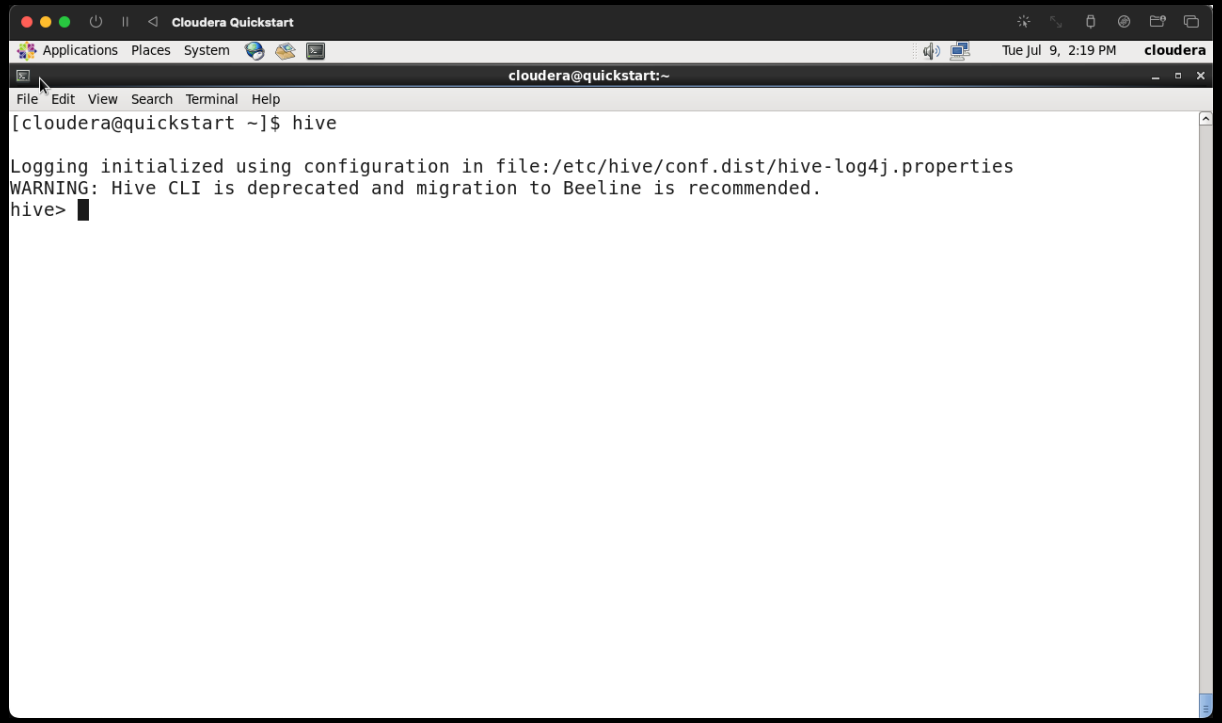
Hive Demo: List Local Data Files



Hive Demo: Command Closeup

- `ll *.txt`

Hive Demo: Enter Hive CLI



Hive Demo: Command Closeup

- `hive`

Hive Demo: Create Words1 Table and Load Words1 Data

Cloudera Quickstart

cloudera@quickstart:~

File Edit View Search Terminal Help

```
hive> create table words1 ( word string );
OK
Time taken: 0.271 seconds
hive> load data local inpath '/home/cloudera/words1.txt' overwrite into table words1;
Loading data to table default.words1
Table default.words1 stats: [numFiles=1, numRows=0, totalSize=64, rawDataSize=0]
OK
Time taken: 1.118 seconds
hive>
```

Hive Demo: Command Closeup

- `create table words1 ( word string );`
- `load data local inpath '/home/cloudera/words1.txt' overwrite into table words1`

Hive Demo: Create Words2 Table and Load Words2 Data

Cloudera Quickstart

cloudera@quickstart:~

File Edit View Search Terminal Help

```
hive> create table words2 ( word string );
OK
Time taken: 0.155 seconds
hive> load data local inpath '/home/cloudera/words2.txt' overwrite into table words2;
Loading data to table default.words2
Table default.words2 stats: [numFiles=1, numRows=0, totalSize=607430, rawDataSize=0]
OK
Time taken: 0.994 seconds
hive>
```

Hive Demo: Command Closeup

- `create table words2 ( word string );`
- `load data local inpath '/home/cloudera/words2.txt' overwrite into table words2`

Hive Demo: Create Words3 Table and Load Words3 Data

Cloudera Quickstart

cloudera@quickstart:~

File Edit View Search Terminal Help

```
hive> create table words3 ( word string );
OK
Time taken: 0.212 seconds
hive> load data local inpath '/home/cloudera/words3.txt' overwrite into table words3;
Loading data to table default.words3
Table default.words3 stats: [numFiles=1, numRows=0, totalSize=5096590, rawDataSize=0]
OK
Time taken: 1.242 seconds
hive>
```

Hive Demo: Command Closeup

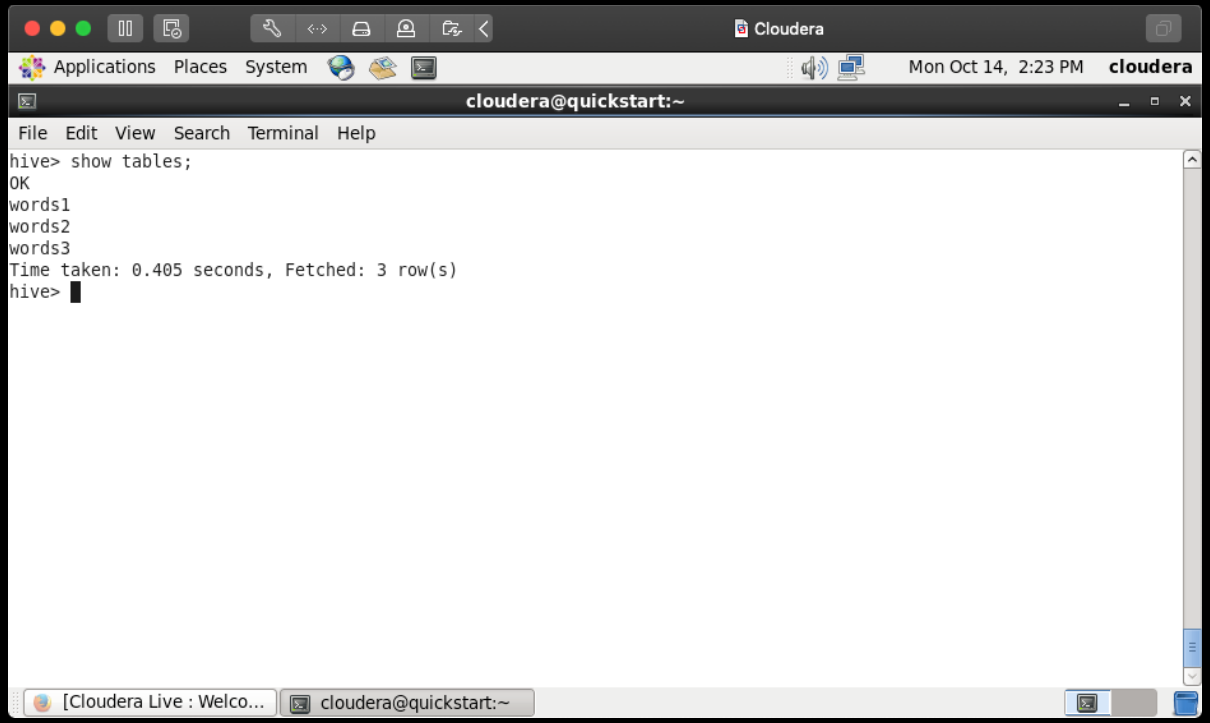
- `create table words3 ( word string );`
- `load data local inpath '/home/cloudera/words3.txt' overwrite into table words3`

Hive Demo: Show All Databases

Hive Demo: Command Closeup

```
show databases;
```

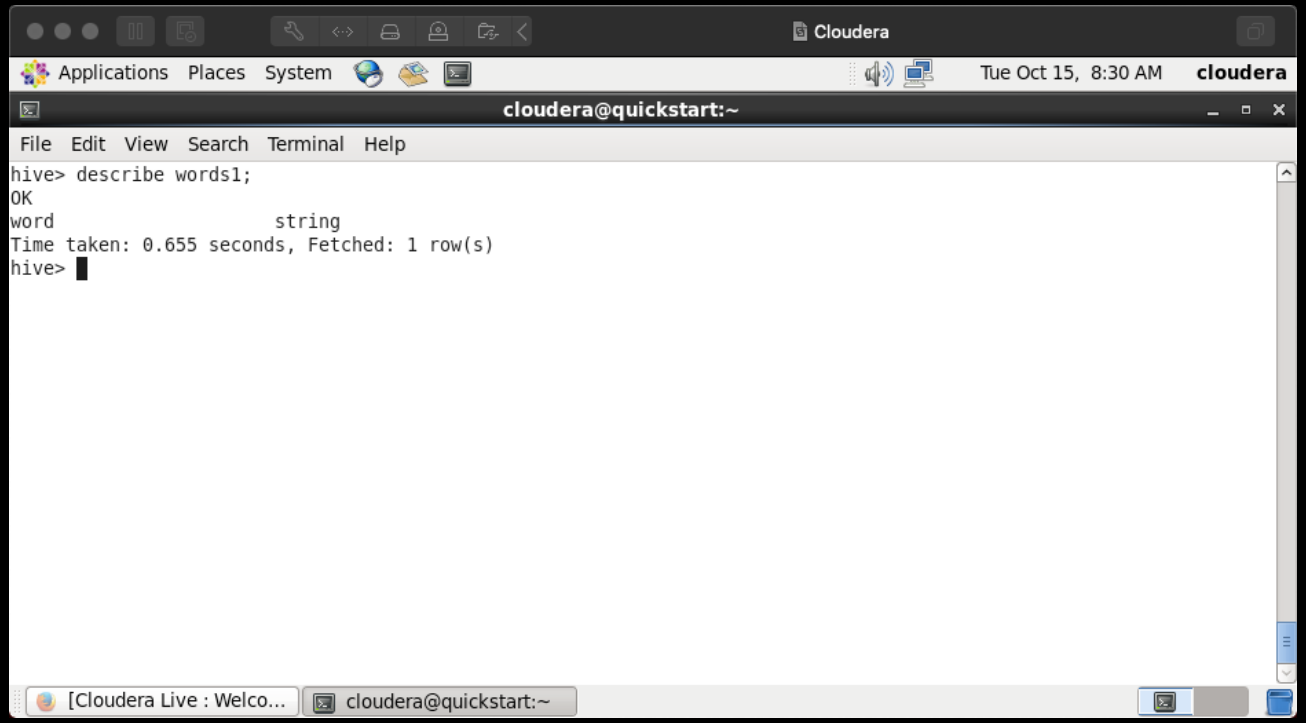
Hive Demo: Show All Tables in Hive



Hive Demo: Command Closeup

```
show tables;
```

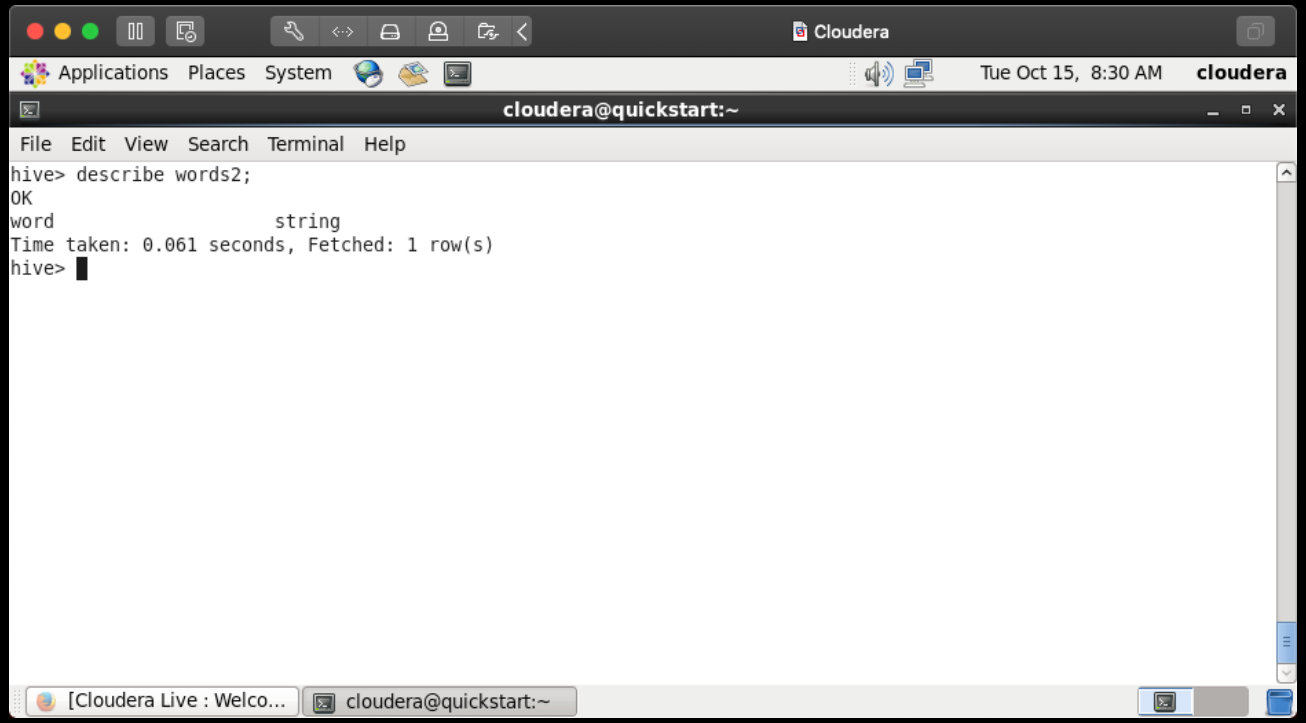
Hive Demo: Describe Words1 Table



Hive Demo: Command Closeup

```
describe words1;
```

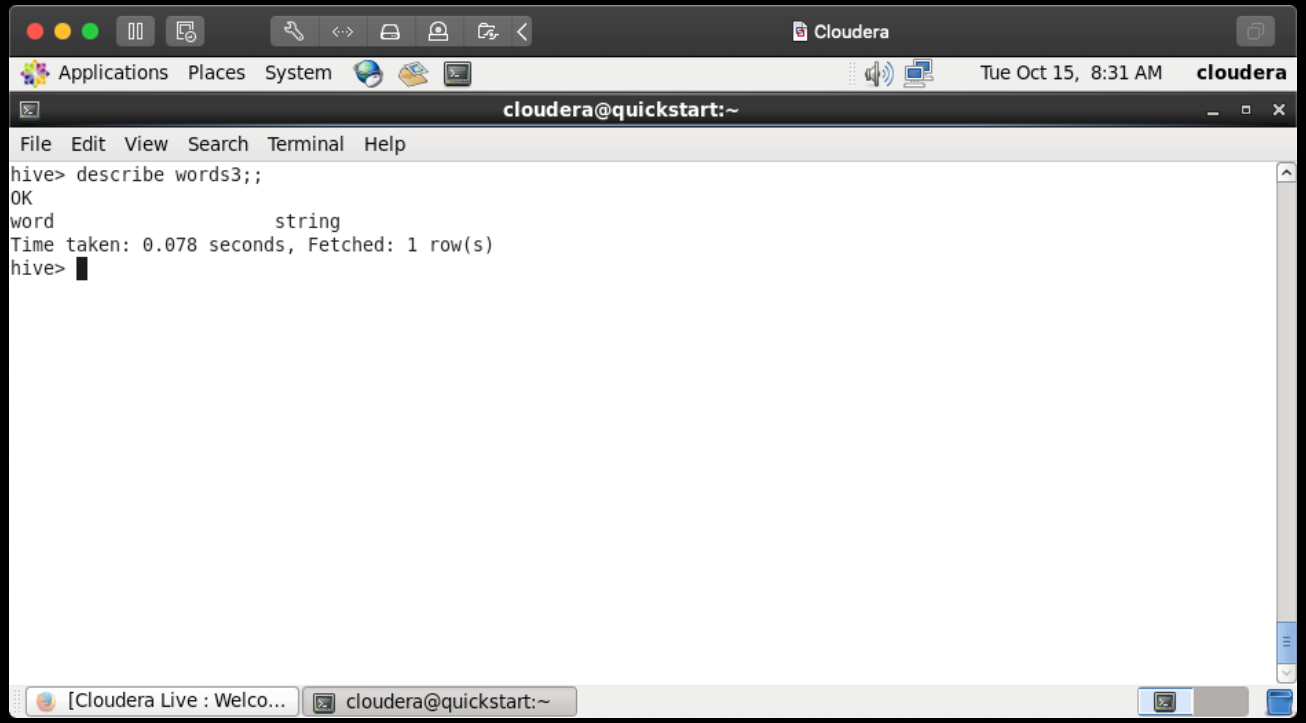
Hive Demo: Describe Words2 Table



Hive Demo: Command Closeup

```
describe words2;
```

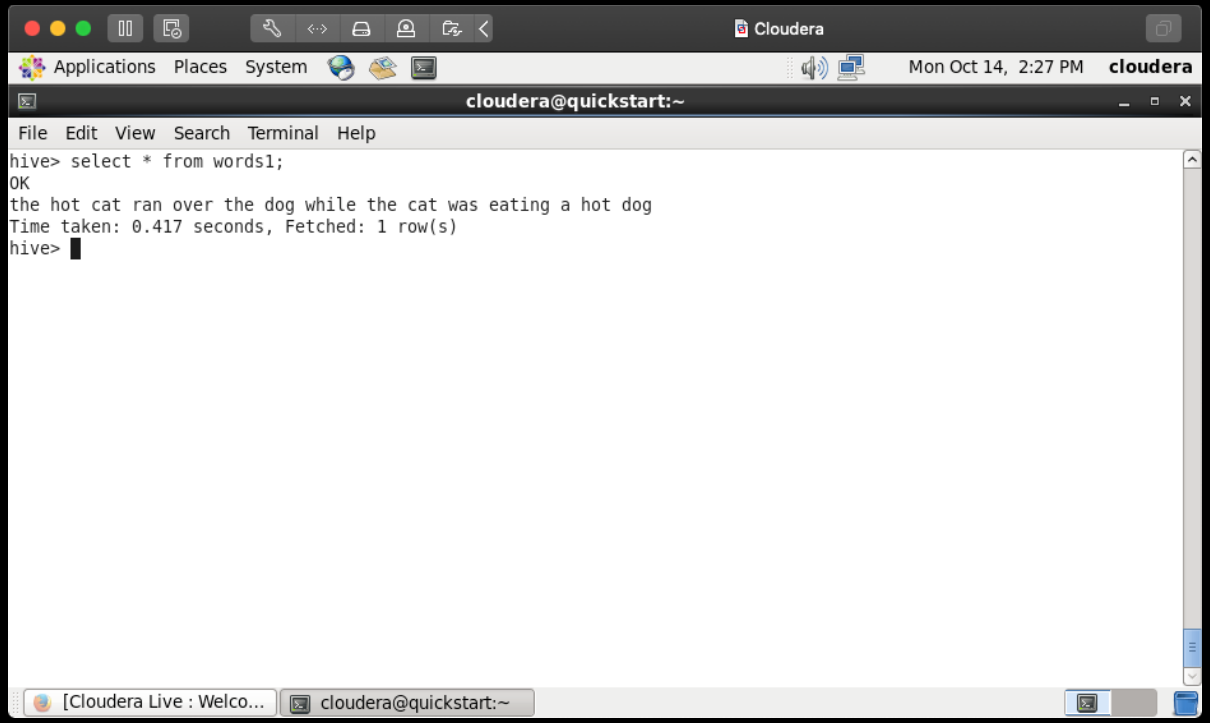
Hive Demo: Describe Words3 Table



Hive Demo: Command Closeup

```
describe words3;
```

Hive Demo: Show Data in Words1



Hive Demo: Command Closeup

```
select * from words1;
```

Hive Demo: Count Rows in Words1

```
hive> select count(*) from words1;
Query ID = cloudera_20191014151010_92c3a8fe-e916-4cc4-ba51-4027b1107933
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1571085335854_0005, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1571085335
854_0005/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1571085335854_0005
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2019-10-14 15:10:24,448 Stage-1 map = 0%,  reduce = 0%
2019-10-14 15:10:32,049 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 0.95 sec
2019-10-14 15:10:38,445 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 2.03 sec
MapReduce Total cumulative CPU time: 2 seconds 30 msec
Ended Job = job_1571085335854_0005
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 2.03 sec   HDFS Read: 7183 HDFS Write: 2 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 30 msec
OK
1
Time taken: 24.49 seconds, Fetched: 1 row(s)
hive>
```

Hive Demo: Command Closeup

```
select count( * ) from words2;
```

Hive Demo: Count Rows in Words2

```
hive> select count(*) from words2;
Query ID = cloudera_20191014151111_7181fd12-f449-4cd2-9597-5894ddc141ec
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1571085335854_0006, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1571085335
854_0006/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1571085335854_0006
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2019-10-14 15:11:10,087 Stage-1 map = 0%,  reduce = 0%
2019-10-14 15:11:18,580 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 1.4 sec
2019-10-14 15:11:25,977 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 2.6 sec
MapReduce Total cumulative CPU time: 2 seconds 600 msec
Ended Job = job_1571085335854_0006
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 2.6 sec   HDFS Read: 6495783 HDFS Write: 7 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 600 msec
OK
128457
Time taken: 24.587 seconds, Fetched: 1 row(s)
hive>
```

Hive Demo: Command Closeup

```
select count( * ) from words3;
```

Hive Demo: Count Rows in Words3

```
hive> select count(*) from words3;
Query ID = cloudera_20191014143232_dc935f6b-e995-4ac2-8759-cb75737031d7
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1571085335854_0004, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1571085335
854_0004/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1571085335854_0004
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2019-10-14 14:32:52,070 Stage-1 map = 0%,  reduce = 0%
2019-10-14 14:32:59,799 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 1.4 sec
2019-10-14 14:33:08,486 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 2.71 sec
MapReduce Total cumulative CPU time: 2 seconds 710 msec
Ended Job = job_1571085335854_0004
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 2.71 sec   HDFS Read: 5103719 HDFS Write: 7 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 710 msec
OK
950184
Time taken: 27.843 seconds, Fetched: 1 row(s)
hive>
```

Hive Demo: Word Count of Words1

```
hive> SELECT word, count(1) AS count FROM ( select explode ( split ( word, ' ' ) ) as word from words1 ) w GROUP B
Y word ORDER BY word;
Query ID = cloudera_20191014152323_347a681d-7381-4c20-b080-5b472da11c09
Total jobs = 2
Launching Job 1 out of 2
Stage-Stage-2: Map: 1  Reduce: 1   Cumulative CPU: 2.01 sec   HDFS Read: 5195 HDFS Write: 64 SUCCESS
Total MapReduce CPU Time Spent: 4 seconds 160 msec
OK
a       1
cat     2
dog     2
eating  1
hot     2
over    1
ran     1
the     3
was     1
while   1
Time taken: 45.914 seconds, Fetched: 10 row(s)
hive>
```

Hive Demo: Command Closeup

```
SELECT word, count( 1 ) AS count \
FROM \
( \
  SELECT explode( split( word, ' ' ) ) AS word \
  FROM words1 \
) temp \
GROUP BY word \
ORDER BY word;
```

Hive Demo: Word Count of Words1

```
hive> SELECT word, count(1) AS count FROM ( select explode ( split ( word, ' ' ) ) as word from words1 ) temp GROU
P BY word ORDER BY count desc;
Query ID = cloudera_20191014153838_c70ade04-fd61-47cf-95b0-895fe022b4bb
Total jobs = 2
[redacted]
Total MapReduce CPU Time Spent: 5 seconds 610 msec
OK
the     3
hot     2
dog     2
cat     2
while   1
was     1
ran     1
over    1
eating  1
a       1
Time taken: 51.581 seconds, Fetched: 10 row(s)
hive>
```

Hive Demo: Command Closeup Words1 Word Count

```
SELECT word, count( 1 ) AS count \
FROM \
( \
  SELECT explode( split( word, ' ' ) ) AS word \
  FROM words1 \
) temp \
GROUP BY word \
ORDER BY count DESC;
```

Hive Demo: Command Closeup Words2 Word Count

```
SELECT word, count( 1 ) AS count \
FROM \
( \
  SELECT explode( split( word, ' ' ) ) AS word \
  FROM words2 \
) temp \
GROUP BY word \
ORDER BY count DESC;
```

Hive Demo: Command Closeup Words3 Word Count

```
SELECT word, count( 1 ) AS count \
FROM \
( \
  SELECT explode( split( word, ' ' ) ) AS word \
  FROM words3 \
) temp \
GROUP BY word \
ORDER BY count DESC;
```

Hive Demo: HQL vs MapReduce Java

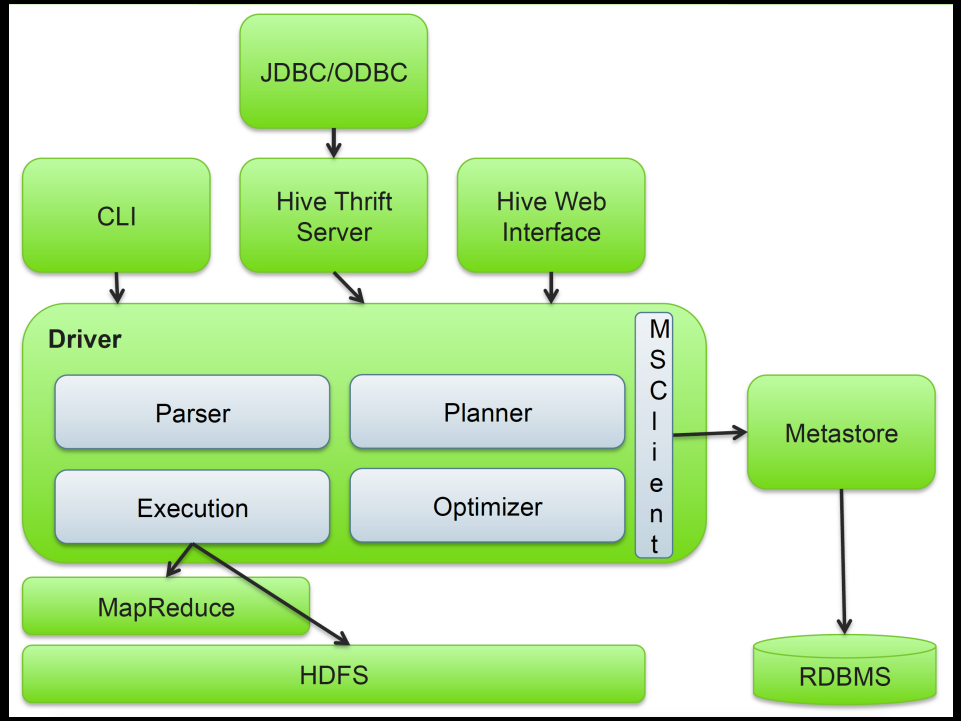
```
SELECT word, count( 1 ) AS count
FROM
(
  SELECT explode( split( word, ' ' ) ) AS word
  FROM words1
) temp
GROUP BY word
ORDER BY count DESC;
```

```
package org.myorg;
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.conf.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
public class WordCount {
  public static class Map extends Mapper < LongWritable, Text, Text, IntWritable > {
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
    public void map(LongWritable key, Text value, Context context)
    throws IOException,
    InterruptedException {
      String line = value.toString();
      StringTokenizer tokenizer = new StringTokenizer(line);
      while (tokenizer.hasMoreTokens()) {
        word.set(tokenizer.nextToken());
        context.write(word, one);
      }
    }
  }
  public static class Reduce extends Reducer < Text, IntWritable, Text, IntWritable > {
    public void reduce(Text key, Iterable < IntWritable > values, Context context)
    throws IOException,
    InterruptedException {
      int sum = 0;
      for (IntWritable val: values) {
        sum += val.get();
      }
      context.write(key, new IntWritable(sum));
    }
  }
  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = new Job(conf, "wordcount");
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    job.setMapperClass(Map.class);
    job.setReducerClass(Reduce.class);
    job.setInputFormatClass(TextInputFormat.class);
    job.setOutputFormatClass(TextOutputFormat.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    job.waitForCompletion(true);
  }
}
```

Exit Hive CLI: “exit;”

```
hive> exit;
WARN: The method class org.apache.commons.logging.impl.SLF4JLogFactory#release() was invoked.
WARN: Please see http://www.slf4j.org/codes.html#release for an explanation.
[cloudera@quickstart ~]$
```

Hive Architecture



Hive Architecture Components

- Metastore
- Driver
- HDFS
- MapReduce
- Clients

Hive Metastore

- The Metastore Stores All Metadata for Hive Database Schemas
- Like the Hadoop NameNode, the Metastore is Critical to Hive and therefore, a Single Point of Failure
- The Metastore is Often Implemented using a Relational Database such as PostgreSQL or MySQL

Hive Driver

- The Core Component of Hive Execution
- Components:
  1. Parser: Checks the Syntax of HQL
  2. Planner: Generates a Query Execution Plan (QEP)
  3. Optimizer: Analyzes the QEP to Improve Performance
  4. Executor: Translates the QEP into MapReduce code and Runs the MapReduce Job

MapReduce

- The Hive Driver ultimately translates HQL into optimized MapReduce Jobs
- Hadoop’s MapReduce Executes Translated HQL Jobs as we observed previously

Hive and HDFS

- Hive Data is Typically Stored in HDFS
- *Remember that Hive Metadata is Stored Separately in the Metastore*
- By Default, Hive Data is in “/user/hive/warehouse”

What is Apache HBase?

- Like Hive, HBase Brings Data Warehousing Capabilities.
- HBase was developed by Powerset, a company developing Natural Language Processing (NLP) Technologies
- Powerset based HBase on Google's "Bigtable" Paper
- HBase Became an Official Apache Project in 2008

Hive versus HBase

- Hive doesn’t change how data is stored in HDFS
- HBase is a “Columnar” Database built on HDFS
- Hive is Read Only
- HBase offers Random Read/Write Access
- Hive supports a simple query language based on SQL
- HBase does not support SQL but has its own non-standard query language

Columnar Databases

- A “Columnar Database” or “Column-Oriented Database”, Stores and Manages Data in Columns rather than Rows
- Most Conventional Databases Store and Manage Data in Rows
- *Conventional Databases are “Row-Oriented Databases”*

Columnar Databases

- Columnar Databases by Design, “Vertically Partition” Data with Respect to How Data is Stored on Disk (Vertical = Column, Horizontal = Row)
- By Storing Data by Column rather than by Row, a Columnar Database Can Process Millions or Billions of Rows of Data far more Efficiently (Faster) than a Conventional Database
- *Data Warehouse are Typically Columnar Databases*

Database Logical Schema Example

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000

Row-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000

Row-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000

Row-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000

Row-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000

Row-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000

Row-Oriented DB Physical Layout

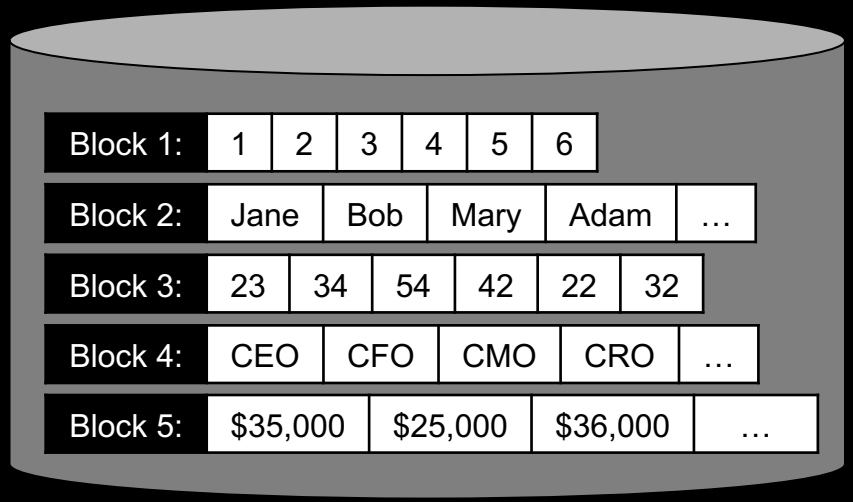
employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000

Row-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000

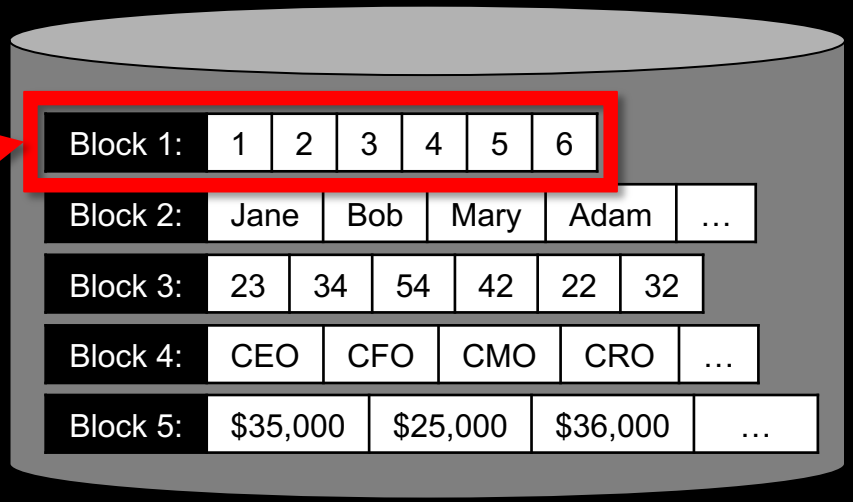
Column-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000



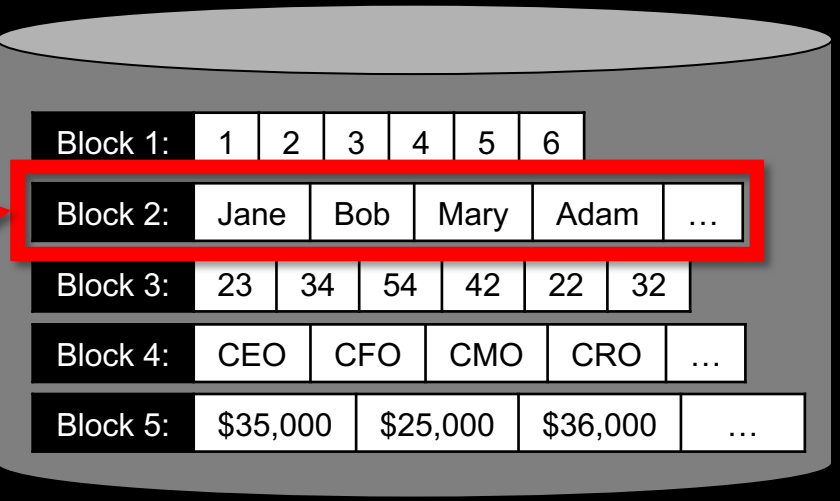
Column-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000



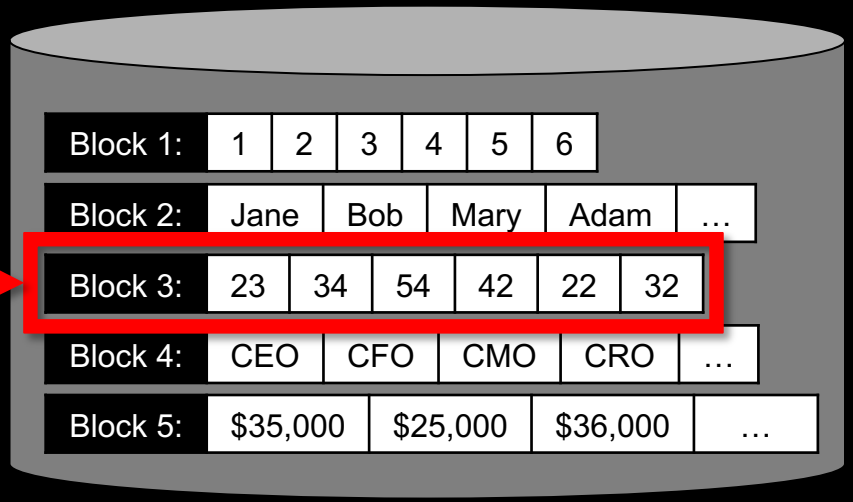
Column-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000



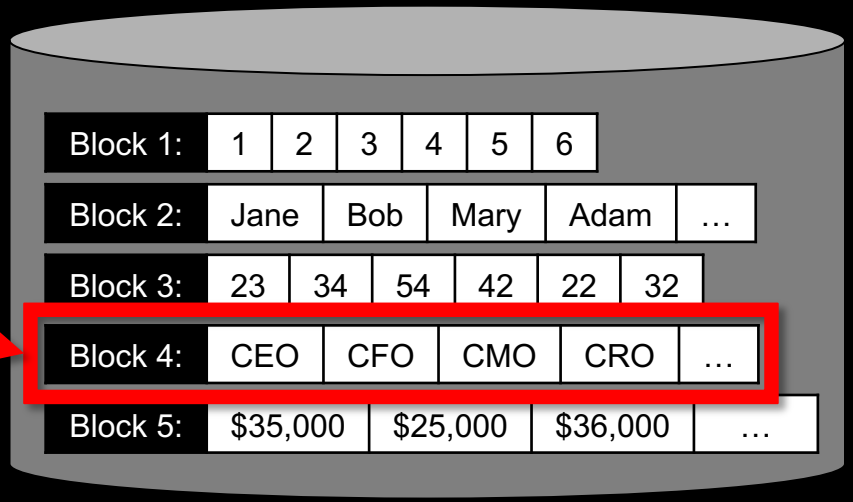
Column-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000



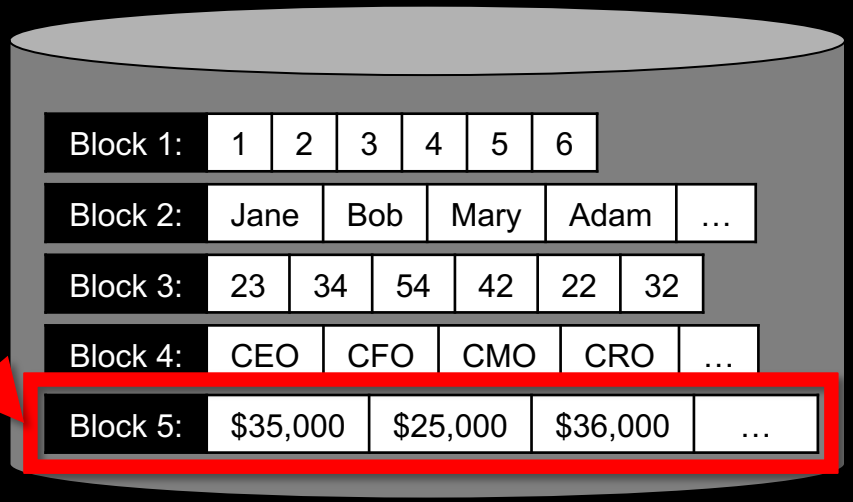
Column-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000



Column-Oriented DB Physical Layout

employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000



Columnar Databases Performance

- Columnar Databases Perform Better because Large Scale Analysis is Typically done Using Entire Table Columns of Data
- Row-Oriented Databases Have to Retrieve Entire Rows only to “Throw Out” Unneeded Data

Hadoop and HDFS Limitations

- Because Hadoop is Batch-Oriented, it's Unsuitable for Stream-Oriented and Real-Time Applications
- Furthermore, HDFS's "Write-Once, Read-Many" Paradigm makes Hadoop Unsuitable for Write Heavy Applications
- Finally, HDFS is Designed for Applications that Process Entire Datasets, and is Unsuitable for Applications that Require Random Access

HBase Random Read/Write Access

- HBase Brings Random Read/Write Access to Hadoop and HDFS by Providing:
  - Fast Lookups to Random Data in Large Tables
  - Low Latency Writes to Random Data in Large Tables

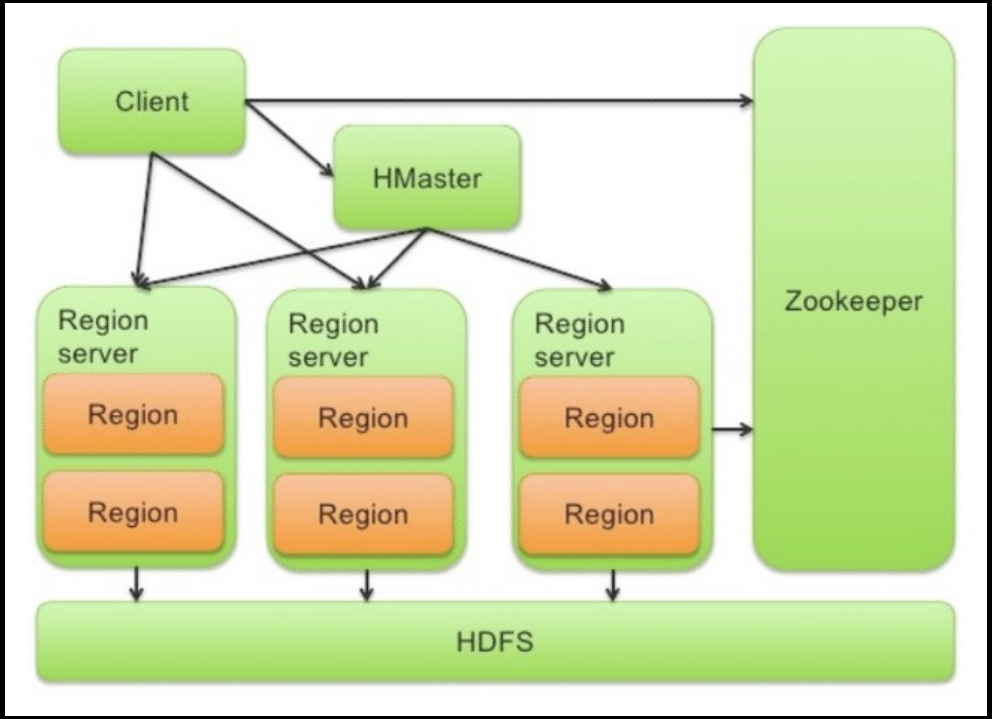
HBase Versus Conventional Databases

- Like Hive, HBase is “Schema on Read” whereas Conventional Databases are “Schema on Write”
- HBase is Designed for Very Wide and Very Tall Tables, up to Millions of Columns and Billions of Rows
- HBase However, does not Provide Transactions, ACID Guarantees, or other Relational Features found in Conventional Databases
- Unlike Conventional Databases, HBase Stores Column Values as Key-Value Pairs

HBase Architecture

- Master Server or “HMaster”
- Region Servers
- Zookeeper
- HDFS
- HBase Clients

HBase Architecture



HBase Master Server or “HMaster”

- HBase’s Equivalent to a NameNode and JobTracker:
  - Administrates the HBase System
  - Coordinates the Region Servers
  - Manages HBase Metadata Data
  - Connection Point for HBase Clients
- HBase’s “Single Point of Failure”
- *Redundant Master Servers run in an “Active-Passive” Configuration*

HBase Region Servers

- HBase's Equivalent to a DataNode and TaskTracker though Data is stored in HDFS
- The Data and Compute Nodes of HBase
- A "Region" is a Subset of the Data in a Table(s)
- A "Region Server" Provides Read and Write Operations to one or more Subsets of Tables
- HBase Automatically "Splits" Data into Regions based on the Data, but this Can be Controlled as Needed

HDFS

- HBase Utilizes HDFS for Storing and Managing Data
- Changes to Data in HDFS are Made Directly by the HBase Region Servers
- *Unlike Hive, HBase does not utilize MapReduce*

HBase Clients

- HBase Shell
- HBase API
- MapReduce
- Hive
- RPC (Thrift and REST)

HBase Data Model

- Tables
- Regions
- Row Keys
- Column Families

HBase Tables

- Tables in HBase are Split and Stored in Subsets across Region Servers
- A “Row Key” Defines an Index into a Region’s Data, so Multiple Row Keys Define Multiple Regions
- Regions therefore provide Fast Access to Data based on the Corresponding Row Key
- *The Process of Distributing Data into HBase Regions is called “Splitting” or sometimes “Sharding”*

HBase Table Row Keys Example

- For Example, Storing Log Data in a HBase, Row Keys can be Defined as Follows:
  - Timestamp
  - Username
  - Product Name
- Defining One or More Row Keys Results in One or More Corresponding Region “Splits” or “Shard”
- *Row Keys are Like “Secondary Indexes” in a Relational Database*

Database Primary and Secondary Indexes

- A “primary index” is an *ordering* of the data in a database table that is based on the physical ordering of the data.
- The ”primary key” is an identifier that uniquely identifies a particular row in a database table, that provides direct/fast access to the data associated with the primary key.
- The set of primary keys are essentially the primary index.
- *Think of the unit numbers in a multi-story building: Unit 101, 212, 324, 415, 536, etc.*

Database Primary and Secondary Indexes

- A “secondary index” is an *additional* ordering of the data in a database table that is independent of the physical ordering of the data.
- A secondary index is defined using one or more columns from a database table and is not necessarily unique.
- The secondary index provides direct access to the data associated with the secondary index but it is not as direct nor fast as a primary key.
- *Think of the lobby directory in a multi-story building: “David the Dentist: 101”, “Adam the Accountant: 212, “Peter the Plumber: 324”, etc.*

Database Primary and Secondary Indexes

HBase Column Families

- A Logical Grouping of Columns in an HBase Table is a “Column Family”
- A Region Always Stores a Column Family Together so Column Families Must be Sized Appropriately
- *Column Families are Defined Based on “Access Patterns”, Logical Groupings of Data that are Typically Needed Together for Processing (Data Locality)*

HBase Column Family Example

Row Key	Column Family 1: “Personal”		Column Family 2: “Professional”	
employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000

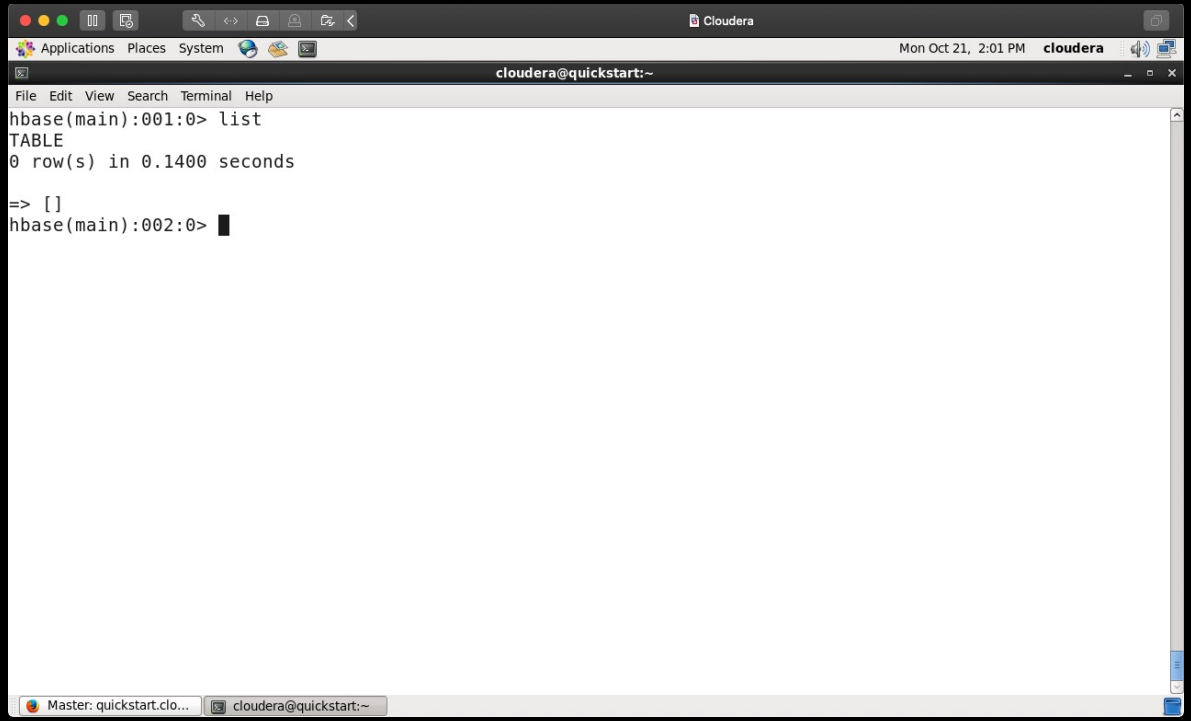
HBase Demo

- The HBase Shell is an Interactive Command Line Interface to HBase
- Unlike Hive, the HBase Shell has its Own Unique Syntax and Conventions
- *The HBase Shell is Clunky :(*

Entering HBase Shell: “hbase shell”

```
[cloudera@quickstart ~]$ hbase shell
2019-10-21 14:00:12,467 INFO  [main] Configuration.deprecation: hadoop.native.lib is deprecated. Instead,
 use io.native.lib.available
HBase Shell; enter 'help<RETURN>' for list of supported commands.
Type "exit<RETURN>" to leave the HBase Shell
Version 1.2.0-cdh5.13.0, rUnknown, Wed Oct  4 11:16:18 PDT 2017

hbase(main):001:0>
```

Listing All Tables: “list”



Creating a Table: “create…”

```
hbase(main):002:0> create 'emp', 'personal data', 'professional data'
0 row(s) in 1.2540 seconds

=> Hbase::Table - emp
hbase(main):003:0>
```

Creating a Table in HBase: “create…”

```
create 'emp', 'personal data',
'professional data'
```

- Table Name is “emp”
- Two Column Families:
  - “Personal Data”
  - “Professional Data”

Listing All Tables: “list”

```
hbase(main):003:0> list
TABLE
emp
1 row(s) in 0.0040 seconds

=> ["emp"]
hbase(main):004:0>
```

Describing a Table: “describe”

```
hbase(main):004:0> describe 'emp'
Table emp is ENABLED
emp
COLUMN FAMILIES DESCRIPTION
{NAME => 'personal data', DATA_BLOCK_ENCODING => 'NONE', BLOOMFILTER => 'ROW', REPLICATION_SCOPE => '0',
VERSIONS => '1', COMPRESSION => 'NONE', MIN_VERSIONS => '0', TTL => 'FOREVER', KEEP_DELETED_CELLS => 'FAL
SE', BLOCKSIZE => '65536', IN_MEMORY => 'false', BLOCKCACHE => 'true'}
{NAME => 'professional data', DATA_BLOCK_ENCODING => 'NONE', BLOOMFILTER => 'ROW', REPLICATION_SCOPE => '
0', VERSIONS => '1', COMPRESSION => 'NONE', MIN_VERSIONS => '0', TTL => 'FOREVER', KEEP_DELETED_CELLS =>
'FALSE', BLOCKSIZE => '65536', IN_MEMORY => 'false', BLOCKCACHE => 'true'}
2 row(s) in 0.0870 seconds

hbase(main):005:0>
```

HBase Column Family Example

Row Key	Column Family 1: “Personal”		Column Family 2: “Professional”	
employee_id	name	age	title	salary
1	Jane	23	CEO	$35,000
2	Bob	34	CFO	$25,000
3	Mary	54	CMO	$36,000
4	Adam	42	CRO	$39,000
5	Carla	22	CTO	$29,000
6	Robert	32	CIO	$32,000

Inserting Data into a Table: “put”

```
hbase(main):014:0> put 'emp','1','personal data:name','Jane'
0 row(s) in 0.0100 seconds

hbase(main):015:0> put 'emp','1','personal data:age','23'
0 row(s) in 0.0030 seconds

hbase(main):016:0> put 'emp','1','professional data:title','CEO'
0 row(s) in 0.0040 seconds

hbase(main):017:0> put 'emp','1','professional data:salary','$35,000'
0 row(s) in 0.0040 seconds

hbase(main):018:0>
```

Inserting Data into a Table: “put”

- `put 'emp','1','personal data:name','Jane'`
- `put 'emp','1','personal data:age','23'`
- `put 'emp','1','professional data:title','CEO'`
- `put 'emp','1','professional data:salary','$35,000'`

Inserting Data into a Table: “put”

```
hbase(main):008:0> put 'emp','2','personal data:name','Bob'
0 row(s) in 0.0070 seconds

hbase(main):009:0> put 'emp','2','personal data:age','34'
0 row(s) in 0.0070 seconds

hbase(main):010:0> put 'emp','2','professional data:title','CFO'
0 row(s) in 0.0060 seconds

hbase(main):011:0> put 'emp','2','professional data:salary','$25,000'
0 row(s) in 0.0060 seconds

hbase(main):012:0>
```

Inserting Data into a Table: “put”

- `put 'emp','2','personal data:name','Bob'`
- `put 'emp','2','personal data:age','34'`
- `put 'emp','2','professional data:title','CFO'`
- `put 'emp','2','professional data:salary','$25,000'`

Listing Data in a Table: “scan”

```
hbase(main):012:0> scan 'emp'
ROW                          COLUMN+CELL
 1                           column=personal data:age, timestamp=1571693290547, value=23
 1                           column=personal data:name, timestamp=1571693287053, value=Jane
 1                           column=professional data:salary, timestamp=1571693299054, value=$35,000
 1                           column=professional data:title, timestamp=1571693294132, value=CEO
 2                           column=personal data:age, timestamp=1571693310298, value=34
 2                           column=personal data:name, timestamp=1571693305839, value=Bob
 2                           column=professional data:salary, timestamp=1571693317253, value=$25,000
 2                           column=professional data:title, timestamp=1571693314029, value=CFO
2 row(s) in 0.0400 seconds

hbase(main):013:0>
```

Listing a Row in a Table: “get”

```
hbase(main):013:0> get 'emp', '1'
COLUMN                          CELL
 personal data:age              timestamp=1571693290547, value=23
 personal data:name             timestamp=1571693287053, value=Jane
 professional data:salary       timestamp=1571693299054, value=$35,000
 professional data:title        timestamp=1571693294132, value=CEO
4 row(s) in 0.0140 seconds

hbase(main):014:0>
```

Listing a Row in a Table: “get”

```
hbase(main):014:0> get 'emp', '2'
COLUMN                          CELL
 personal data:age              timestamp=1571693310298, value=34
 personal data:name             timestamp=1571693305839, value=Bob
 professional data:salary       timestamp=1571693317253, value=$25,000
 professional data:title        timestamp=1571693314029, value=CFO
4 row(s) in 0.0240 seconds

hbase(main):015:0>
```

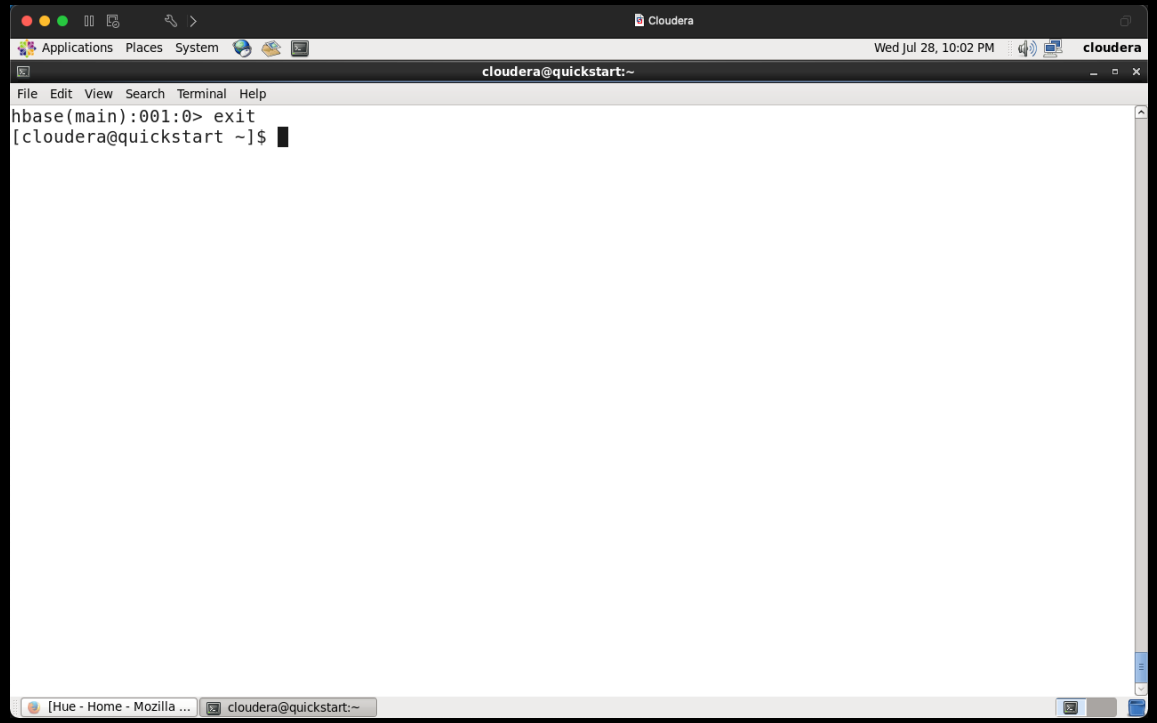
Updating Data in a Table: “put”

```
hbase(main):022:0> put 'emp','1','professional data:salary','$45,000'
0 row(s) in 0.0060 seconds

hbase(main):023:0>
```

Listing a Row in a Table: “get”

```
hbase(main):002:0> get 'emp', '1'
COLUMN                          CELL
 personal data:age              timestamp=1571693290547, value=23
 personal data:name             timestamp=1571693287053, value=Jane
 professional data:salary       timestamp=1571693732430, value=$45,000
 professional data:title        timestamp=1571693294132, value=CEO
4 row(s) in 0.0120 seconds

hbase(main):003:0>
```

Exiting HBase Shell: “exit”



Homework

- Readings:
  - “Schema-on-Read vs Schema-on-Write”
  - “Hive - A Petabyte Scale Data Warehouse Using Hadoop”
  - “Bigtable - A Distributed Storage System for Structured Data”
  - “ColumnStores vs. RowStores - How Different are They Really”
  - “HBase - A NoSQL Database”
- Complete Lecture 4 Homework
- Review Lecture 4 and Prepare for Test

Big Data Terminology Recap

- Hive
- Data Warehouse
- Online Analytical Processing (OLAP)
- Online Transactional Processing (OLTP)
- HiveQL (HQL)
- Command Line Interface (CLI)
- Schema on Write
- Schema on Read
- Metastore
- Driver
- Parser
- Planner
- Optimizer
- Executor
- Query Execution Plan (QEP)
- ODBC
- JDBC
- HBase
- Columnar or Column-Oriented Database
- Row-Oriented Database
- HBase Master or “HMaster”
- Region Servers
- Data “Splitting” or “Sharding”
- Row Keys
- Column Families

Thanks!

Questions or feedback?

Email Me via Canvas

UCLA Extension